

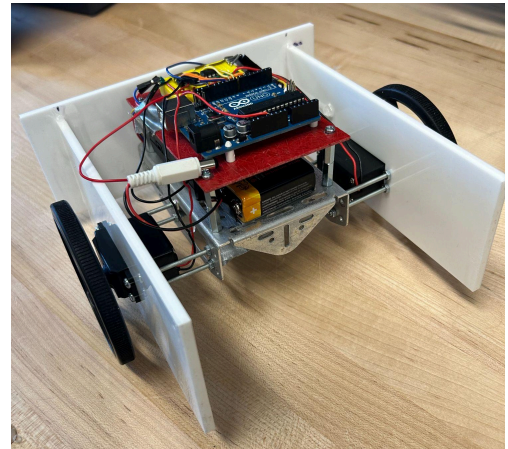
Final Report

MAE 3780, Group 5

Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

Robot Design and Strategy Overview

The mechanical design of our robot focuses on optimizing the internal space available for collecting cubes. The chassis is elevated by larger wheels that sit on the outside of a laser-cut white acrylic perimeter, as shown on the right. The perimeter forms three sides of a roughly 8 x 8 inch square. The usable cube-collecting area is slightly smaller than the maximum 8 x 8 inch limit to accommodate the width of the wheels and a reserved section in the back of the chassis for the color sensor. However, the design still provides sufficient space to collect and trap blocks while the robot drives forward.



The robot's electrical system consists of two L9110 H-bridges, two DC wheel motors, an Arduino Uno, and a color sensor. Each H-bridge controls one DC motor. The output pins of the H-bridges are connected to the positive and negative terminals of the motors, while the input pins are connected to the digital output pins on the Arduino. By changing the input signals, the Arduino controls the direction of the wheel rotation, and thus the overall robot motion. The robot also uses a color sensor connected as an input to the Arduino. The H-bridges are powered by a 6V (4 AA batteries in series), while the Arduino is powered separately by a 9V battery. These are stored under the mounted Arduino and breadboard to conserve space and maintain stability.

The software design is based on open-loop control and consists of motor functions as well as an interrupt based on the color sensor's reading. The motor functions include `drive_forward()`, `turn_left()`, and `stop()`. These functions operate by setting the appropriate H-bridge input pins to control the relative movement of the wheels. Setting both pins high or low stops the motor. Setting one input high and the other low causes the motor to rotate in one direction, while reversing the signals reverses the direction. To process color information, the robot uses a timer interrupt to measure the frequency output of the color sensor. The robot stores the detected period corresponding to the initial surface color, and then turns when the measured period changes by more than a calibrated difference threshold of 100, which indicates a switch from blue to yellow or vice-versa. Additionally, if the period exceeds a calibrated absolute threshold corresponding to the color black, the robot stops.

During competition, the robot is set at a roughly 30 degree angle from the back black border, facing rightward. It deploys after plugging in the 9V battery to the Arduino. Then, ideally, the robot drives forward, detects the blue-yellow border and turns left. The degree of the turn is pre-calibrated by adjusting the delay after the `turn_left()` function such that the robot is oriented parallel to the blue-yellow border. Then, it drives forward, collecting blocks until it reaches the black border and stops. In practice, this was not achieved due to friction from the blocks, leading to asymmetrical drag that caused the robot to rotate or stall. Since our software is open-loop, it does not allow for direction correction.

Design Process Reflection

A description of your group's design/prototyping/testing process throughout the course of the project. How did your group progress through the design process and achieving the milestones? Did you run into any significant roadblocks, need to change the design of your robot, etc.?

We started the design process by meeting up and discussing ideas we had for -wheel issue, doesn't drive straight

Competition Analysis

A summary of your robot's performance on competition day. How did your robot perform in the tournament? What worked and what didn't work? Did you encounter anything unexpected (good or bad)? What were the strengths and weaknesses of your robot?

Conclusions

After reflecting on our competition performance, design, and build, we identified several areas of improvement. Overall, we believe our robot to be successful, as we met all the core requirements and collected a significant amount of cubes. However, as we watched our robot compete, we recognized how we could improve our sensor selection and logic.

If we were to redo this project with the same constraints and budget, one of the first changes we would make is implementing a “turn and move away from the black border” response rather than a “stop on the black border” response. Our code made it so that if our robot detected the black border after it had already performed one turn, it would stop. Although this made our code simple to implement, it prevented our robot from maximizing cube collection. There were rounds where our robot detected the black border very early in the match, due to being pushed by the opposing robot. This resulted in our robot being stationary for the majority of the round. We would edit our code so that the robot recognizes the black border, reverses, and turns away from the boundary, into the middle of the board, so it can resume collecting cubes. This change

would increase the amount of time that the robot is moving, thus allowing us to maximize the number of cubes collected.

Another change we would make is to remove the delay at the beginning of our code. This delay caused our robot to take a few seconds to move after the match had started. Throughout each round, we realized that speed is a significant advantage. There were matches where we were unable to collect cubes due to the opposing robot blocking our path. This change would allow us to access cubes and approach an optimal path before the opposing robot got in the way.

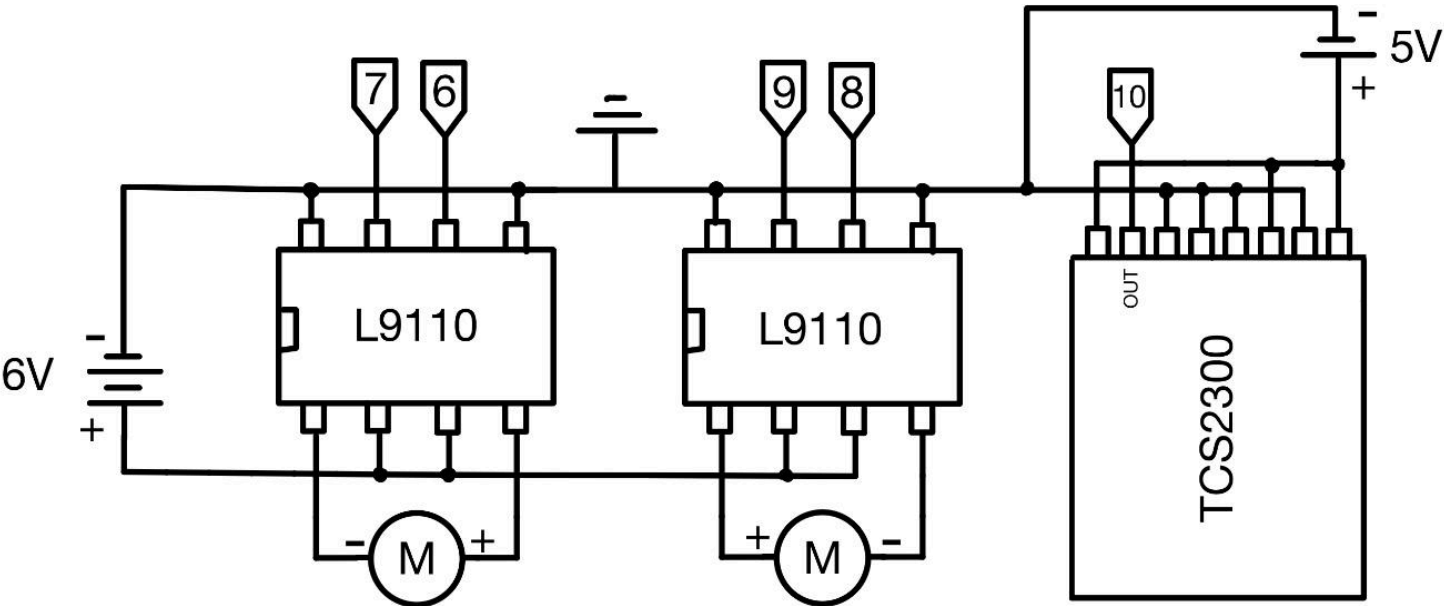
Our final, and most complex, change would be to implement QTI sensors into our robot. This would allow our robot to detect not only that it had encountered the border, but specifically which side our robot was contacting it. Making this change would give our robot an added directional awareness that would allow it to always turn back into the center of the playing area. Our lack of QTI sensors is what led us to make the decision to simply stop when it detected black. With our code, our robot had to start each round in the exact same orientation. QTI sensors would allow us to start our robot in any orientation, giving us a spatial advantage.

For students doing this project next year, I would advise them to be bold and not play it safe. My team and I played it safe with a simple design and simple logic. Although our robot was successful, we believe that we would have made it further in the competition if we had taken chances on more complex implementations. A robot that adapts and reacts is always going to dominate over a robot that pauses, hesitates, or stops.

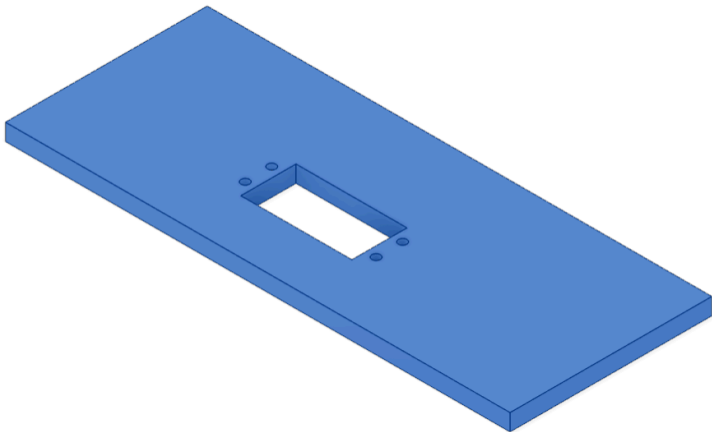
Appendix A: Bill of Materials

Item Name	Unit Price	Quantity	Item ID	Vendor	Additional Manufacturing Cost	Total Cost
Wheels	\$2	2	N/A	From lab	N/A	\$4
Structural Adhesive, Waterproof Epoxy, J-B Weld Clearweld, 0.5 FL.oz Tube	\$7.11	1	7605A25	McMaster	N/A	\$7.11
Steel Pan Head Phillips Screw, M3 x 0.5 mm Thread, 50 mm Long	\$7.32	1	92005A137	McMaster	N/A	\$7.32
Acrylic Sheet	\$5	1	N/A	RPL	0.5*3 + 0.05*64	\$4.70
TOTAL COST						= 23.13

Appendix B: Circuit Diagram



Appendix C: CAD Files and Drawings





PROPERTIES

Bodies (1)

Area

31663.549 mm^2

Density

0.001 g / mm^3

Mass

102.689 g

Volume

86438.804 mm^3

Physical Material

Acrylic

Appearance

Plastic - Glossy (Grey)

► Bounding Box

Center of Mass

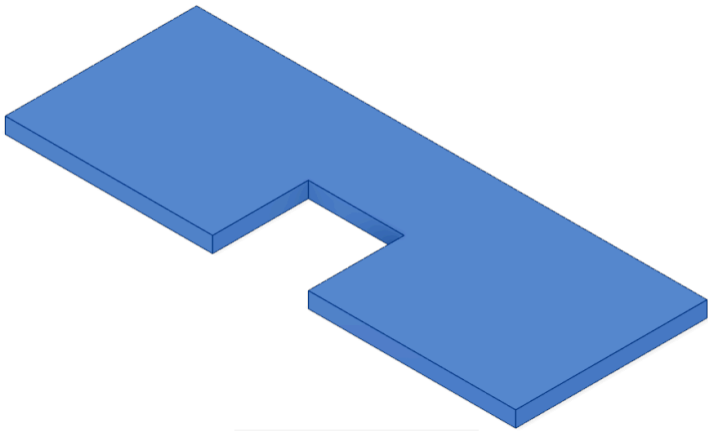
95.63 mm, 38.59 mm, 5...

► Moment of Inertia at Center of Mass (g L...

OK

Cancel

Body2





PROPERTIES

Bodies (1)

Area

49.75 in^2

Density

0.687 ouncemass / in^3

Mass

3.734 ouncemass

Volume

5.438 in^3

Physical Material

Acrylic

Appearance

Plastic - Glossy (Grey)

► Bounding Box

Center of Mass

4.00 in, 1.578 in, 0.125 ...

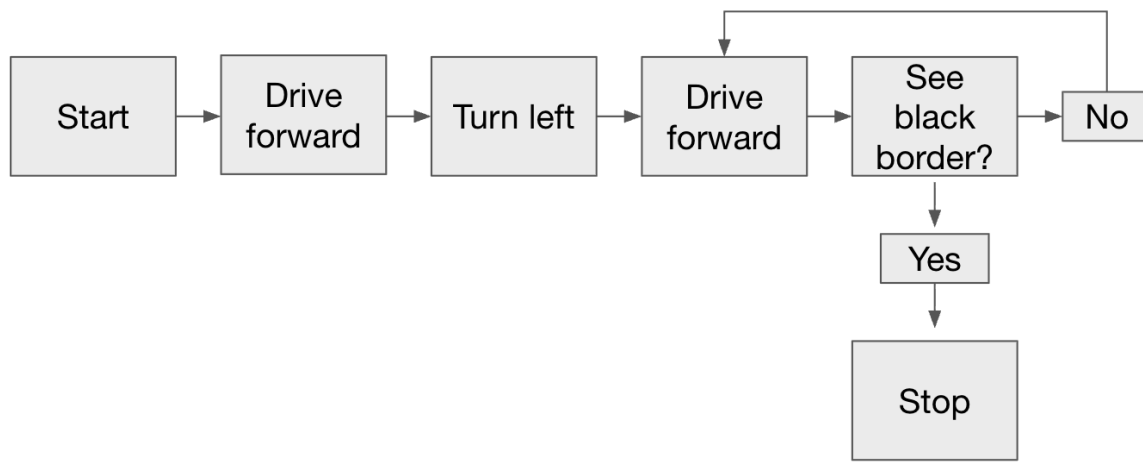
► Moment of Inertia at Center of Mass (ou...

OK

Cancel

Body1

Appendix D: Flowchart



Appendix E: Code

```
// Global Variables
volatile unsigned int timerVal;
int startPeriod;
int blackThreshold = 300; // Calibrated for Black
int colorDiffThreshold = 100; // Difference between Blue and Yellow

// ISR: Tracks the signal from the color sensor on Pin 10 (PB2)
ISR(PCINT0_vect) {
    // Check Pin 10 (bit 2 of Port B)
    if (PINB & 0b00000100) {
        TCNT1 = 0; // Reset timer on rising edge
    } else {
        timerVal = TCNT1; // Capture counts on falling edge
    }
}

// initColor: Setup for Pin 10 and Timer 1. Referenced from pre-lab 4
void initColor() {
    // Set Pin 10 (PB2) as input
    DDRB &= ~0b00000100;

    // Enable Pin Change Interrupt 0 (covers PB0-PB7)
    PCICR |= 0b00000001;

    // Timer 1: Normal mode, Prescaler 1
    TCCR1A = 0;
    TCCR1B = 0b00000001;
    sei(); // Enable global interrupts
}

// getColor: Samples the sensor frequency and returns the period
int getColor() {
    PCMSK0 |= 0b00000100; // Enable interrupt for Pin 10
    _delay_ms(15); // Sampling window
    PCMSK0 &= ~0b00000100; // Disable interrupt
    return (2 * timerVal / 16);
}

// Motor Functions
void drive_forward() {
```


Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

```
PORTB = (PORTB & ~0b00000001) | 0b00000010; // Pin 8 Low, Pin 9 High
PORTD = (PORTD & ~0b01000000) | 0b10000000; // Pin 6 Low, Pin 7 High
}

void turn_left() {
  PORTB = (PORTB & ~0b00000001) | 0b00000001; // Pin 9 Low, Pin 8 High
  PORTD = (PORTD & ~0b01000000) | 0b10000000; // Pin 6 Low, Pin 7 High
}

void stop() {
  PORTB &= ~0b00000001; // Clear pins 8 & 9
  PORTD &= ~0b11000000; // Clear pins 6 & 7
}

// Competition Algorithm
int main(void) {
  Serial.begin(9600);
  DDRB |= 0b00000011; // set pins 8 and 9 as outputs for "left" motor
  DDRD |= 0b11000000; // set pins 6 and 7 as outputs for "right" motor
  initColor();

  // Detect Initial Color
  _delay_ms(1000);
  startPeriod = getColor();

  // Drive until the other color is reached
  drive_forward();
  while (1) {
    int current = getColor();

    if (abs(current - startPeriod) > colorDiffThreshold && current < blackThreshold) {
      stop(); //check again to make sure
      if (abs(current - startPeriod) > colorDiffThreshold && current < blackThreshold) {
        break;
      }
    }
  }
}

// turn
_delay_ms(100);
turn_left();
_delay_ms(500);
```

```
stop();
```

```
// drive until Black Edge
```

```
_delay_ms(600);
```

```
drive_forward();
```

```
while(1) {
```

```
    if (getColor() > blackThreshold) {
```

```
        stop(); // Stop immediately at the edge
```

```
        break;
```

```
    }
```

```
}
```

```
while(1);
```

```
//end
```

```
}
```

Appendix F: Photos

